

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Traditional MapReduce stores intermediate results on disk after every processing step, which makes it slower for large-scale and iterative computations. It also requires writing multiple Map and Reduce jobs for complex workflows. Apache Spark overcomes these limitations by processing data in memory, reducing disk I/O, and supporting faster execution for data analytics, machine learning, and real-time processing.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Spark stores intermediate data in memory instead of writing it to disk after every operation. This allows repeated computations to access the same data much faster, making iterative machine learning algorithms significantly faster than traditional disk-based systems like MapReduce.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

```
# Remove duplicate records based on user_id and transaction_date
```

```
df = df.dropDuplicates(["user_id", "transaction_date"])
```

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

```
from pyspark.sql.functions import avg
```

```
df_sales.filter(df_sales["region"] == "West") \

    .groupBy("product_category") \

    .agg(avg("sale_amount").alias("Average Sale")) \

    .show()
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

`.na.drop()` removes rows containing null values, whereas `.na.fill()` replaces null values with a specified value without deleting the rows.

```
df = df.na.fill({"status": "Unknown"})
```

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
from pyspark.sql.functions import count
```

```
df.groupBy("city") \
```

```
  .agg(count("*").alias("Total Records")) \
```

```
  .filter("Total Records > 100") \
```

```
  .show()
```

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are immutable, which means the original DataFrame cannot be modified directly. Whenever we drop or rename columns, Spark creates a new DataFrame containing the updated changes while keeping the original DataFrame unchanged.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
from pyspark.sql.functions import col
```

```
df.filter(
```

```
    (col("age").between(18, 30)) &
```

```
    (col("subscription") == "Premium")
```

```
).show()
```

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Handling null values before aggregation helps produce accurate results. If null values are ignored or left untreated, calculations like averages and totals may become incorrect or incomplete.

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

```
from pyspark.sql.functions import col

from pyspark.sql.types import TimestampType

df = df.withColumn(

    "event_time",

    col("raw_timestamp").cast(TimestampType())

).drop("raw_timestamp")
```

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Shuffle is the process of redistributing data between partitions during operations like `groupBy()` and `join()`. It is called a wide transformation because data moves across different partitions, requiring network communication before the final result is generated.

Q12: Write a code snippet that identifies and **removes rows** where the `email` column contains null values OR the `username` is an empty string.

```
from pyspark.sql.functions import col

df = df.filter(

    col("email").isNull() &

    (col("username") != "")

)
```

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the `price` column?

```
from pyspark.sql.functions import min, max, avg
```

```
df.agg(

    min("price").alias("Minimum Price"),

    max("price").alias("Maximum Price"),

    avg("price").alias("Average Price")

).show()
```

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

If the date formats are inconsistent, `inferSchema=true` may assign the wrong data type or treat dates as strings. This can lead to incorrect sorting, filtering, and date calculations, so it is often better to define the schema manually when the data quality is poor.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.

```
from pyspark.sql.functions import sum
```

```
result = (

    df.dropDuplicates()

    .fillna({"price": 0})

    .groupBy("store_id")

    .agg(sum("price").alias("Total Revenue"))

)

result.show()
```